05 - Context Compiler Spec

Visor DOM-to-Agent Context Compiler
DOC-05 | 0.1 requirements pack | Generated 2026-06-14

This specification defines the compiler pipeline that converts PageSnapshot into AgentContext using normalization, scoring, dedupe, token budgeting, and export formatting.

Field	Value
Owner	Rohan PX / Visor project
Audience	AI coding agent, reviewer, future contributors
Status	Draft baseline for MVP build
Decision rule	MVP is local-first, minimal-permission, Chrome Manifest V3, no backend unless explicitly enabled.

Document control

Item	Value
Project	Visor DOM-to-Agent Context Compiler
Document code	DOC-05
Generated on	2026-06-14
Primary goal	Turn raw webpage DOM into safe, structured, agent-ready context.
MVP boundary	Chrome extension, local-only compiler, JSON/Markdown export, privacy redaction, tests.

1. Compiler purpose

The compiler transforms a PageSnapshot into AgentContext. It is responsible for normalization, noise filtering, content scoring, grouping, token budgeting, privacy handoff, and output formatting. It must be deterministic and testable.

2. Compiler pipeline

Step	Function	Output
Normalize	Clean whitespace, normalize URLs, trim blocks, canonicalize labels.	NormalizedSnapshot
Dedupe	Hash repeated text, navigation, sidebars, repeated cards.	Deduped blocks with removed-token estimate
Score	Assign importance score to each block.	ScoredBlock[]
Classify	Infer page type and content roles.	PageClassification
Group	Build hierarchy by heading, proximity, and DOM order.	Content sections
Budget	Fit content to selected token budget.	Selected blocks and compression notes
Format	Produce AgentContext and export strings.	JSON, Markdown, prompt block

3. Importance scoring

Signal	Score effect	Rationale
Inside main/article/role=main	Strong positive	Likely main content.
H1/H2/H3	Strong positive	Structure and task intent.
Form label/button/input	Strong positive	Actionable context.
Table with headers	Strong positive	Dense data.
Code block in docs page	Positive	Important for technical pages.
Footer/nav/sidebar repeated text	Negative	Usually noise.
Cookie/ad/newsletter class names	Strong negative	Low value and privacy noise.
Tiny repeated text	Negative unless action label	Often decorative.

4. Output modes

Mode	Goal	Keep	Drop/compress
compact	Smallest useful context.	Summary, key headings, top content, actions.	Long boilerplate, secondary links, low-score blocks.
detailed	Preserve page structure.	Most content sections, tables, forms, links.	Only obvious noise.

Mode	Goal	Keep	Drop/compress
agent_action	Help agent act on UI.	Buttons, forms, labels, error/status text, links.	Long article text unless nearby action.
rag	Chunk for retrieval.	Text chunks with metadata and source path.	Action-only metadata unless useful.
debug	Explain compiler decisions.	Kept blocks, removed blocks, scores, warnings.	Nothing silently hidden.

5. Token budgeting

The compiler should use an approximate token estimator. A safe MVP heuristic is characters divided by 4 for English-like text, with configurable fallback. The exact tokenizer can be upgraded later. Token budgeting is used for prioritization, not billing.

```
type TokenProfile = {
  rawEstimatedTokens: number;
  compiledEstimatedTokens: number;
  removedNoiseTokens: number;
  compressionRatio: number;
  budget: number;
  budgetStatus: 'under_budget' | 'near_budget' | 'over_budget_trimmed';
};
```

6. Context safety labeling

- All webpage text must be placed under data fields, never merged into system/developer prompts.
- Exports must include a warning comment that page content is untrusted.
- Prompt block export must separate instructions from page data using explicit delimiters.
- Actionable elements need confidence scores because labels/selectors can be ambiguous.

7. Compiler acceptance requirements

ID	Requirement	Priority	Acceptance signal
COMP-001	Same input snapshot and settings produce stable AgentContext excluding timestamps.	P0	Determinism test passes.
COMP-002	Compact mode is smaller than detailed mode on long pages.	P0	TokenProfile shows lower compiled tokens.
COMP-003	Action mode preserves forms/buttons even when text budget is low.	P0	Action fixture includes action elements.
COMP-004	Debug mode lists removed blocks and reasons.	P1	Debug fixture contains compiler notes.
COMP-005	RAG mode emits chunks with source URL, heading path, and block IDs.	P1	RAG fixture validates chunk metadata.

Source and reference notes

These sources are used to ground platform/security assumptions. The implementation must still verify current platform behavior during development.

Reference	Why it matters
OWASP LLM Top 10 - LLM01 Prompt Injection: https://genai.owasp.org/llmrisk/llm01-prompt-injection/	Grounding for treating webpage content as untrusted data in agent pipelines.